

Welcome to the Labs

Candy Catcher



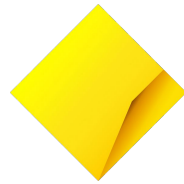
Thank you to our Sponsors!

Platinum Sponsors:



Australian Government
Australian Signals Directorate

Gold Sponsor:



**Commonwealth
Bank**

Who are the tutors?

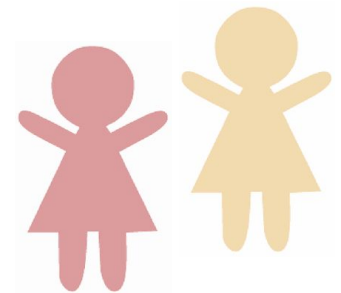
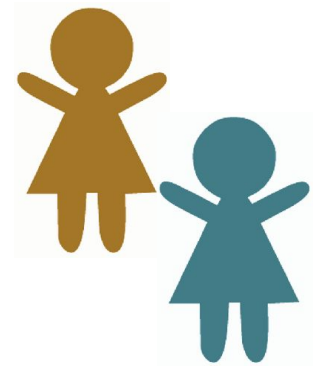


Who are you?



Two Truths and a Lie

1. Get in a group of 3-5 people
2. Tell them three things about yourself:
 - a. Two of these things should be true
 - b. One of these things should be a lie!
3. The other group members have to guess which is the lie



Log on

Log on and jump on the GPN website

girlsprogramming.network/workshop

Click on your node location

Click on your room.

From this page you can see:

- These **slides** (to take a look back or go on ahead).
- A digital copy of your **workbook**.
- Other helpful bits to use through the day!

Tell us you're here!

Click on the
Start of Day Survey
and fill it in now!



Start of Day Survey

Today's project!

Candy Catcher



Candy Catcher

- You're going to build a game called Candy Catcher using Python and Pygame Zero
- Move your hero around the screen with your keyboard to collect as much candy as you can
- Collect all the candy and avoid the bombs to win!



Introduction to Edstem



Log on

Click on your **Workbook** link to take you into EdStem

Workbook

Slides



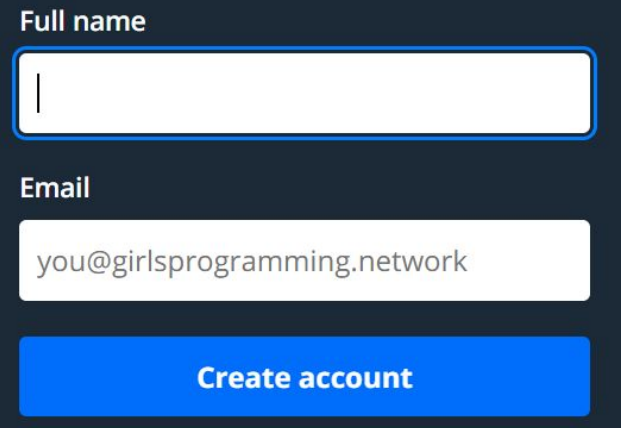
Signing up to Edstem

Log in if you already have a an “Edstem” account from a past GPN

Already have an account? [Log in](#)

If you haven't got an account, let's make one:

1. Type in your Full Name
2. Type in your personal email
3. Click Create Account
4. Go to your email and verify your new account
5. Create a password



A screenshot of the Edstem account creation form. It has a dark blue background. At the top, it says 'Full name' above a white text input field. Below that, it says 'Email' above another white text input field containing the text 'you@girlsprogramming.network'. At the bottom, there is a large blue button with the text 'Create account' in white.

Click Join Course

Join course

The name of your course will be at the top : —————>

Guess Who G

If you don't have access to your email account, ask a tutor for a GPN Edstem login

Getting to the lessons

1. Once you are in the course, you'll be taken to a discussion page.
2. Click the button for the lessons page (top right - looks like a book)



The set up of the workbook

The main page:

1. Heading at the top that tells you the project you are in
2. List of “Chapters” called something like **1: Who are you?**
They have an icon that looks like this: 
3. To complete your project, work through the chapters one at a time



Inside a Chapter

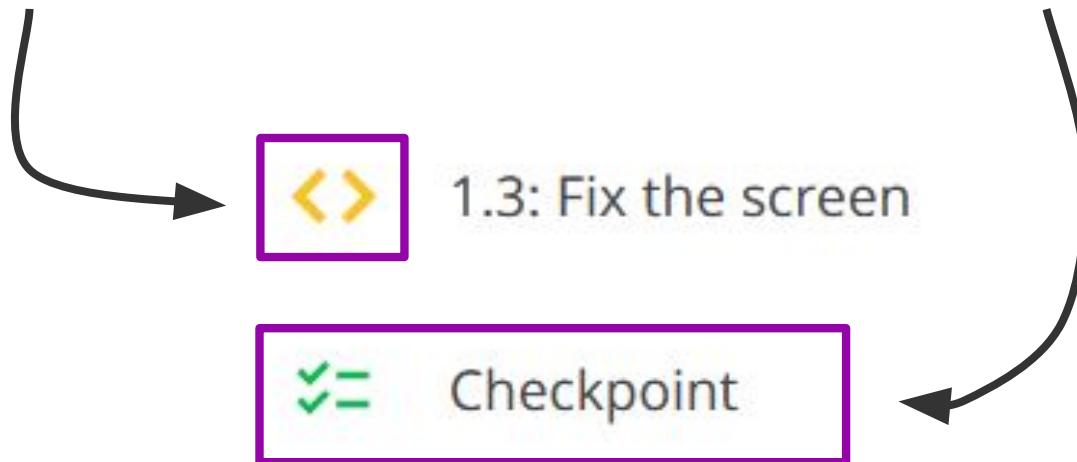
Inside a Chapter there are two main types of pages:

1. Lesson pages

The lessons are where you will do your coding. They have this icon:

2. Checkpoints

Complete the checkpoint to move to the next chapter. Make sure you scroll down to see all the questions listed.



How to do the work

In each lesson there is:

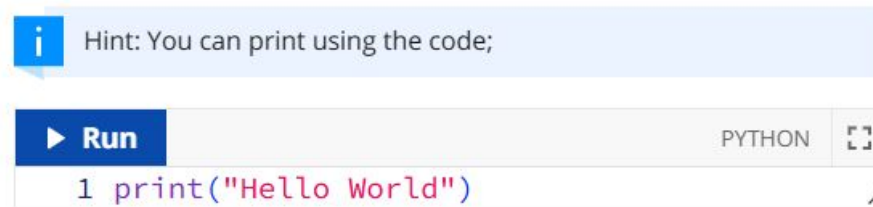
1. A section on left with instructions for that lesson
2. A section on right for your code

You will need to **copy your code from the last lesson**, then follow the instructions to change your code.

The screenshot displays a web-based programming environment. On the left, a 'Description' panel titled 'Example Lesson Page' provides instructions: 'This is an example lesson page. This is where you're instructions and hints will go in each of your lessons.' It also includes a section 'To test out how to write code in this new online workbook...' with two steps: '1. First, print "Hello EdStem!"' and '2. Then run your code in the terminal. Remember you will need to enter the code `python chatbot.py`'. Below this is a 'Hint' box stating 'Hint: You can print using the code;' and a 'Run' button. At the bottom of the description panel is a code input field containing `1 print("Hello World")`. On the right, a code editor window titled 'chatbot.py' shows two lines of Python code: `1 # Put your testing code here!!!` and `2 print("Hello EdStem!")`. The bottom of the interface features a terminal window with the prompt `/home/chatbot.py` and the text 'Spaces: 4 (Auto)' and 'All changes saved'.

Hints

Sometimes in a lesson, there's some code we want you to do that might be a bit tricky, to help you out we've added some hints. They look like this:



If you press the blue run button it will show you what that code does, you can even change the code to see if how it changes.

These are **just hints** make sure you're not copying the hint into your code as it will likely end up breaking. They are just to show you the kinds of things you can do.

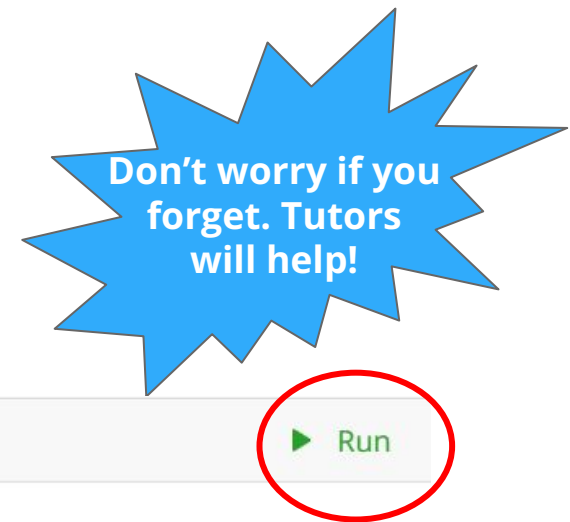
Running your code...

To run your code, click



Your code will run and any output will display in the Console.
You can click the button again to rerun your code.

It should look like this;



Some shortcuts...

There are a couple things you can do to make copying your code from one page to another easier.

- 1) **Ctrl + A** Pressing these keys together will select all the text on a page
- 2) **Ctrl + C** Pressing these keys together will copy anything that's selected
- 3) **Ctrl + V** Pressing these keys together will paste anything you've copied

On Macs use Command (⌘) instead of Ctrl

Intro to PyGame Zero

Making it into a game!



What is PyGame Zero?

We use PyGame Zero to allow our code to do some cool things.



PyGame Zero Setup

The first thing we need to do to use PyGame Zero is to write this at the top of your file

```
import pgzrun
```

PyGame Zero Setup

The first thing we need to do to use PyGame Zero is to write this at the top of your file

```
import pgzrun
```

Now to make sure PyGame Zero runs our code we also need another line at the end of our code. It will make sure all the things run at the right time

```
pgzrun.go()
```

Some Pygame Zero basics

Here's some of the basics of Pygame Zero that you'll need for your game.

Screen:

A game screen will pop up in a window when you run your code.

You can create a screen by setting its size dimensions using keywords WIDTH and HEIGHT

- To make a screen 100 pixels wide x 200 pixels high you would code

```
WIDTH = 100
```

```
HEIGHT = 200
```

You must click on the screen before you start to play your game.



Project time!

You now know all about the basics of EdStem
and PyGame Zero!

Let's put what we learnt into our project
Try to do Part 1

The tutors will be around to help!

PyGame Zero images

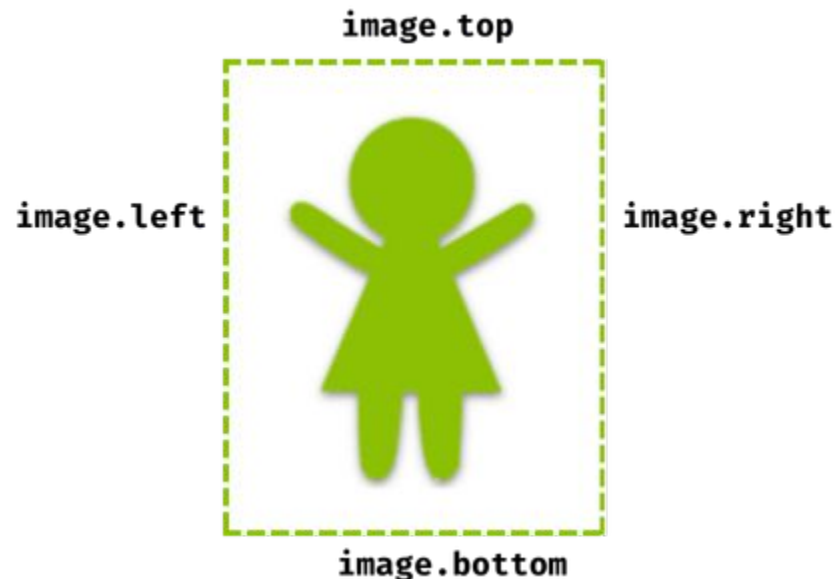


Adding things to our screen!

Images in Pygame zero

Images in Pygame zero are called **Actors**

This is because you can make them move around and do things like actors in a play. Pygame zero stores some information about each of the actors in our game like their position on the screen and what image the actor is.

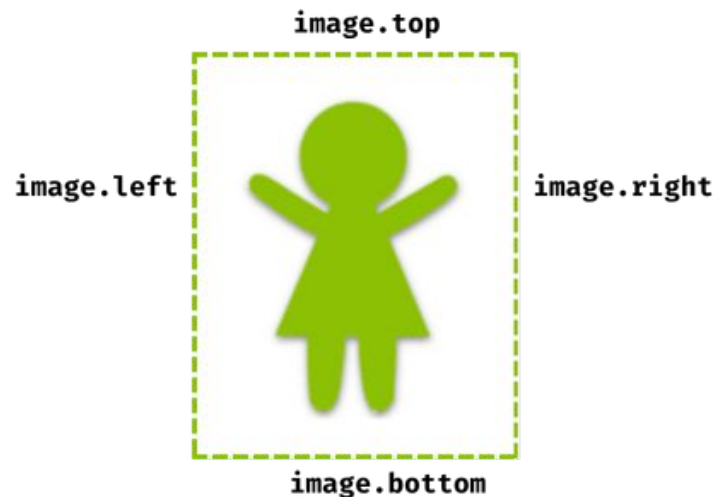


How to make an actor

To make a new actor and tell Pygame zero what image it is you need to write the code:

```
myActor = Actor("myImage")
```

Here the name of our actor is **myActor** and if we need to change anything about it we have to use its name



How to make an actor

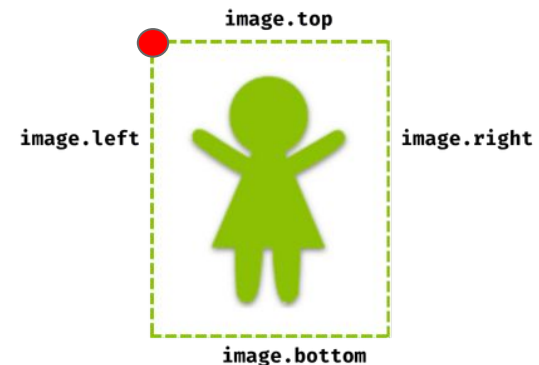
To make a new actor and tell Pygame zero what image it is you need to write the code:

```
myActor = Actor("myImage")
```

Here the name of our actor is **myActor** and if we need to change anything about it we have to use its name

To set our actor's position you use the code:

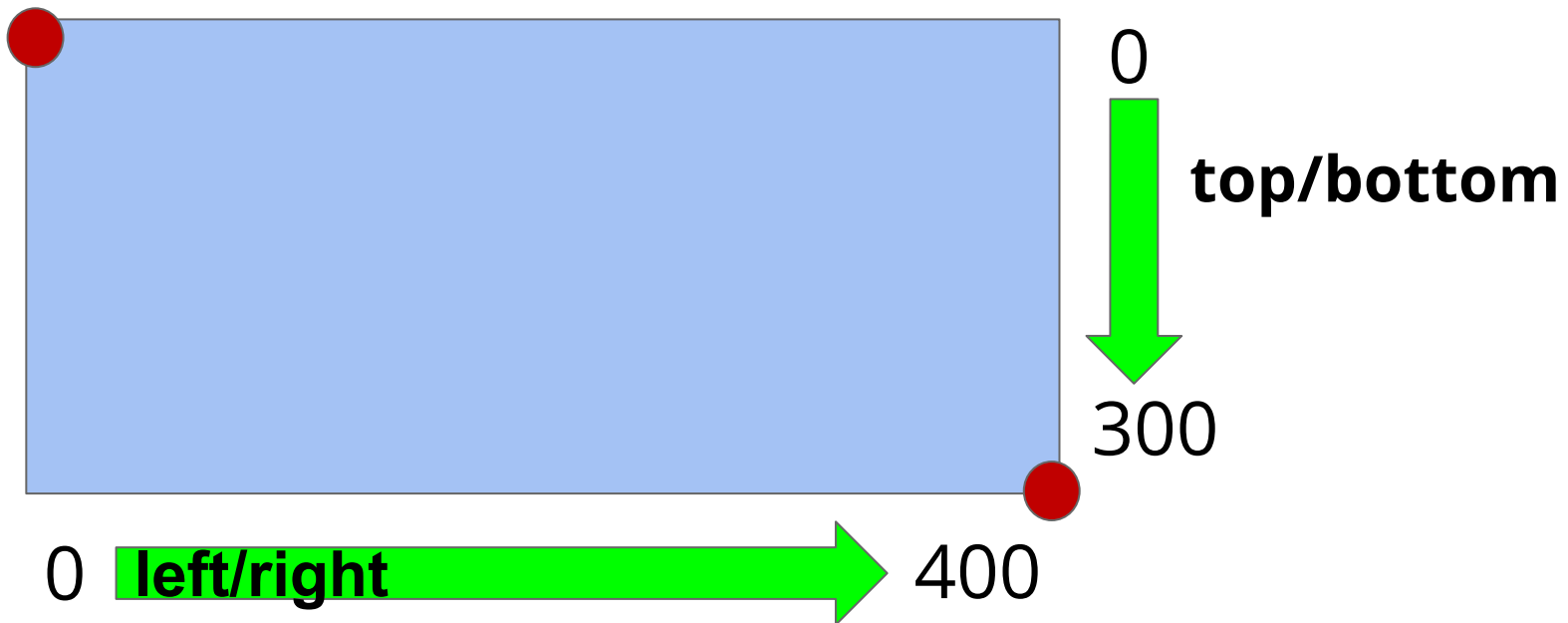
```
myActor.top = 50  
myActor.left = 50
```



- Sets top left hand corner of image 50 pixels down from the top of the screen and 50 pixels away from the left of the screen

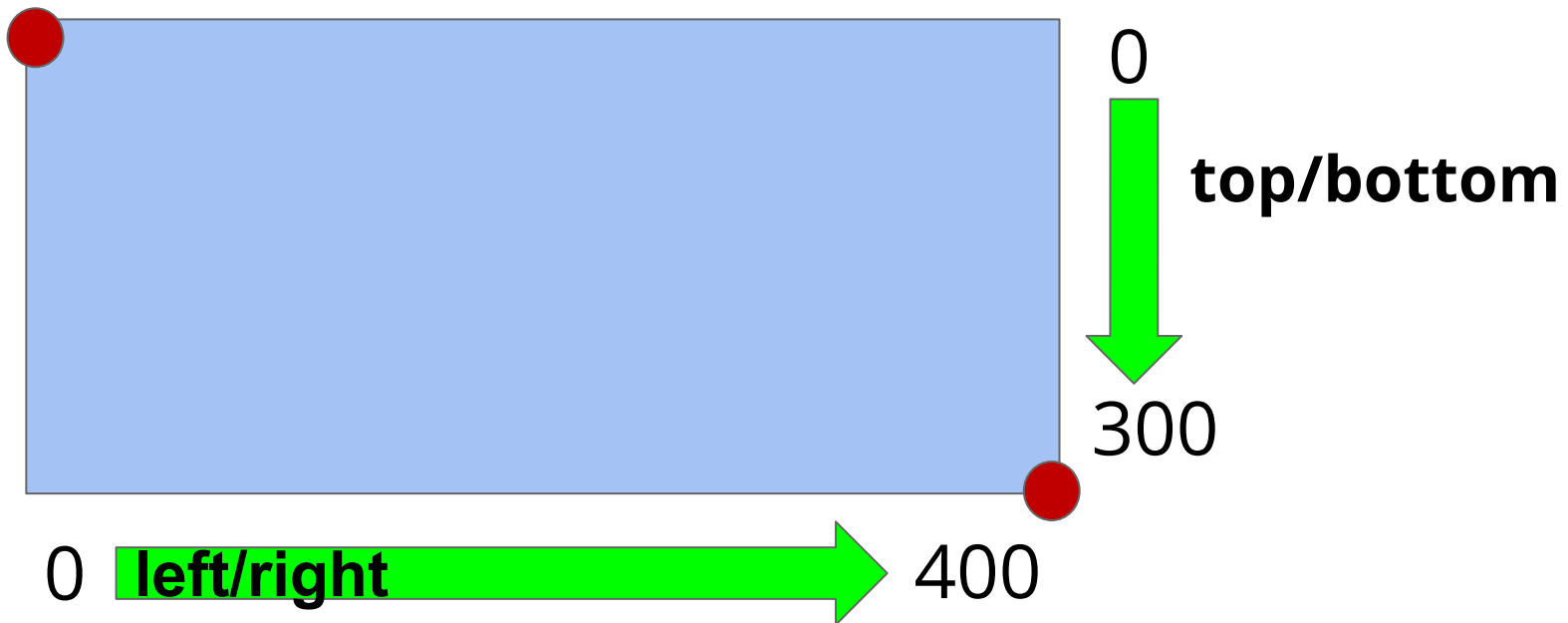
What does the screen look like?

- **left/right** properties increase from left to right (like normal)
- **top/bottom** properties **increase from top to bottom (not normal)**



How to position an actor on the screen

- Change the actor's top/bottom/left/right to move them around the screen
- To move left, decrease left. To move right, increase right.
- To move up, **decrease top**. To move down, **increase bottom**.



Functions

- Sometimes in code we make a **function**.
- A **function** is a set of code that we want to run over and over again.
- It's like a recipe - Instead of writing the same instructions over and over again, you write them once, give them a name, and use that whenever you want to bake!
- **Functions** help us organise our code and make it easier to understand

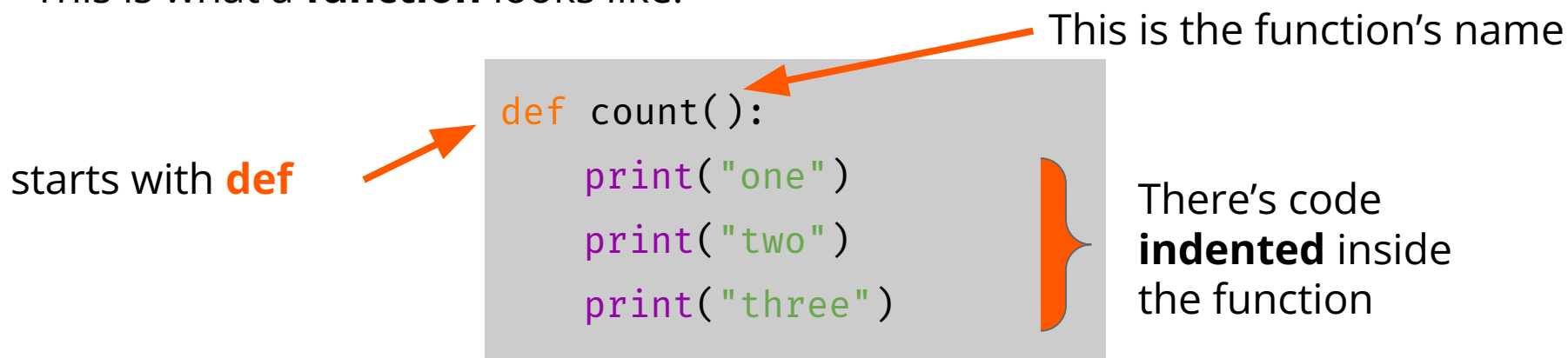
This is what a **function** looks like:

starts with **def**

```
def count():  
    print("one")  
    print("two")  
    print("three")
```

This is the function's name

There's code **indented** inside the function

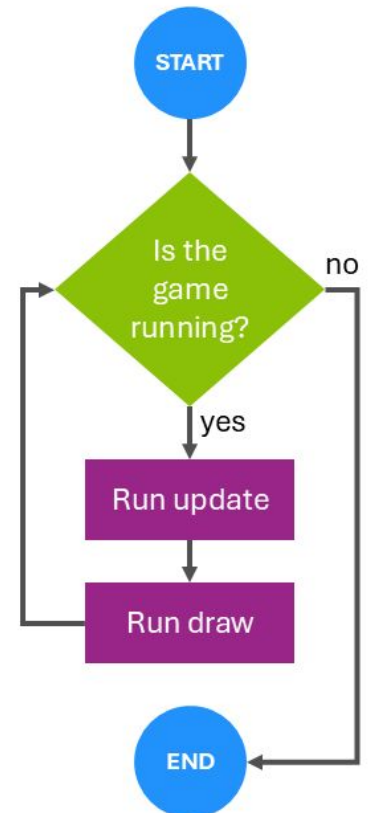
A diagram illustrating the structure of a Python function. It shows a code block with the following text: `def count():` on the first line, followed by three indented lines: `print("one")`, `print("two")`, and `print("three")`. An orange arrow points from the text "starts with def" to the `def` keyword. Another orange arrow points from the text "This is the function's name" to the `count()` part of the first line. A large orange curly brace on the right side of the code block groups the three indented lines, with an arrow pointing to the text "There's code indented inside the function".

PyGame Zero Functions

- PyGame Zero needs two functions: **update** and **draw**
- It runs these functions over and over.

```
def update():  
    # This function is to change things every frame
```

```
def draw():  
    # This function is to add things to the screen every frame
```



Some important code

Pygame zero needs some pretty specific things in order to make our game work. To do these there are three main functions:

```
def draw():  
    # This function is to add things to the screen every frame  
  
def update():  
    # This function is to change things every frame  
  
def on_mouse_down():  
    # This function's code runs every time the player clicks their mouse
```

Project time!

You now know all about Actors!

**Let's see if we can put what we learnt into
our project**

Try to do Part 2

The tutors will be around to help!

If Statements



Conditions!

Conditions let us make decision.
First we test if the condition is met!
Then maybe we'll do the thing



If it's raining take an umbrella

Yep it's raining

..... take an umbrella

Conditions

So to know whether to do something, they find out if it's **True**!

```
fave_num = 5  
if fave_num < 10:  
    print("that's a small number")
```

Conditions

So to know whether to do something, they find out if it's **True**!

```
fave_num = 5  
if fave_num < 10:  
    print("that's a small number")
```

What do you think happens?

Conditions

So to know whether to do something, they find out if it's **True**!

```
fave_num = 5
if fave_num < 10:
    print("that's a small number")
```

What do you think happens?
that's a small number

Conditions

How about a different number???

```
fave_num = 9000
```



```
if fave_num < 10:
```

```
    print("that's a small number")
```

What do you think happens?

Conditions

How about a different number???

```
fave_num = 9000
```



```
if fave_num < 10:
```

```
    print("that's a small number")
```

What do you think happens?

Nothing!

If statements

```
word = "GPN"  
if word == "GPN":  
    print("GPN is awesome!")
```

What happens?

If statements

```
word = "GPN"  
if word == "GPN":  
    print("GPN is awesome!")
```

What happens?

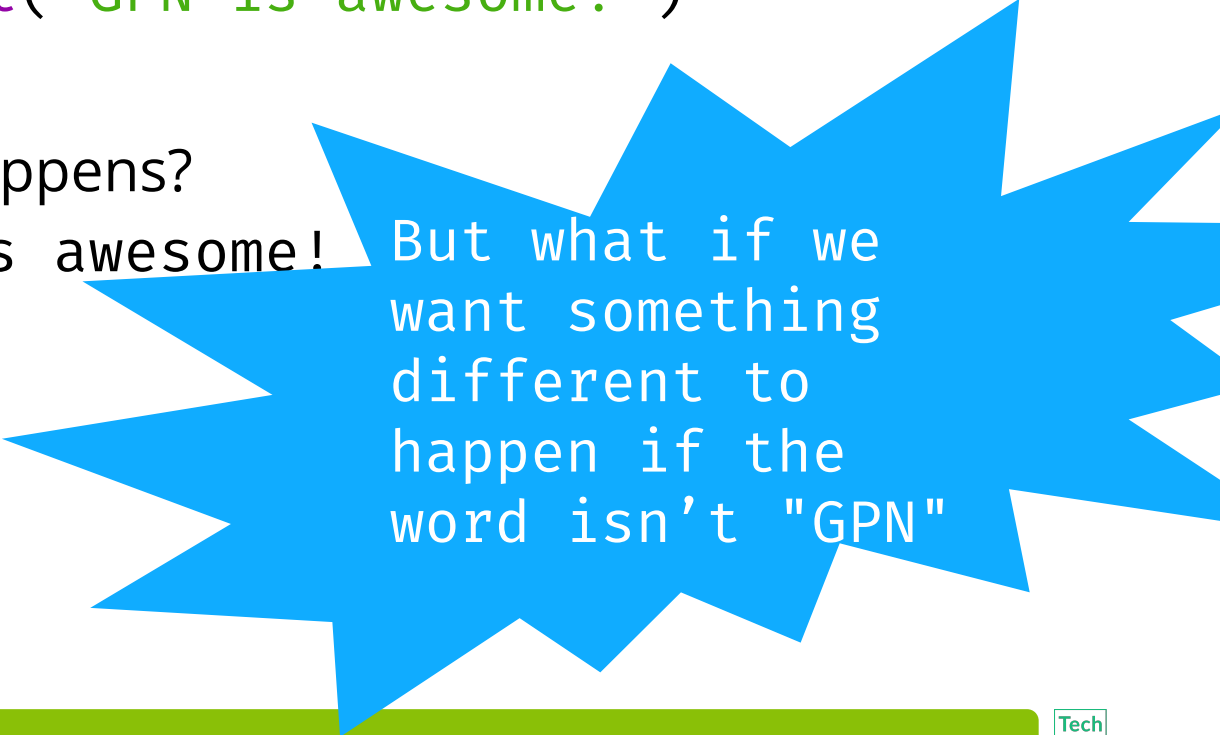
GPN is awesome!

If statements

```
word = "GPN"  
if word == "GPN":  
    print("GPN is awesome!")
```

What happens?

GPN is awesome!



But what if we
want something
different to
happen if the
word isn't "GPN"

Else statements

else
statements means
something still
happens if the **if**
statement was
False

```
word = "Chocolate"  
if word == "GPN":  
    print("GPN is awesome!")  
else:  
    print("The word isn't GPN :(")
```

What happens?

Else statements

else
statements means
something still
happens if the **if**
statement was
False

```
word = "Chocolate"  
if word == "GPN":  
    print("GPN is awesome!")  
else:  
    print("The word isn't GPN :(")
```

What happens?

The word isn't GPN :(

Elif statements

elif

Means we can give specific instructions for other words

```
word = "Chocolate"
if word == "GPN":
    print("GPN is awesome!")
elif word == "Chocolate":
    print("YUMMM Chocolate!")
else:
    print("The word isn't GPN :(")
```

What happens?

Elif statements

elif

Means we can give specific instructions for other words

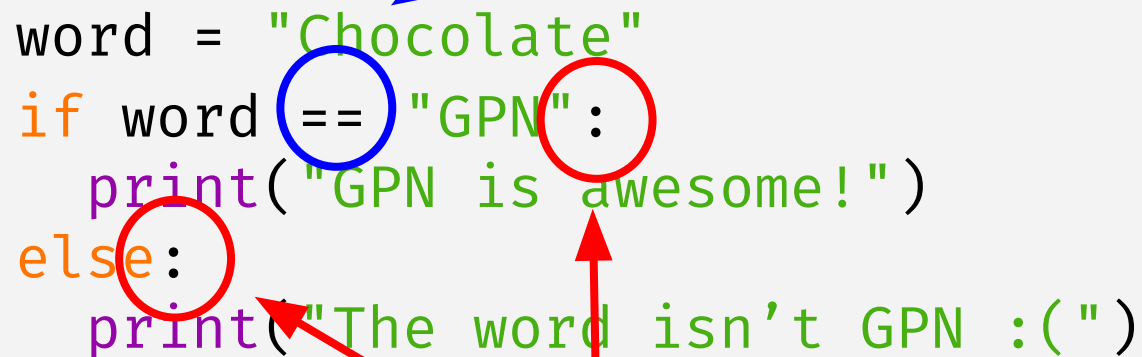
```
word = "Chocolate"
if word == "GPN":
    print("GPN is awesome!")
elif word == "Chocolate":
    print("YUMMM Chocolate!")
else:
    print("The word isn't GPN :(")
```

What happens?
YUMM Chocolate!

== AND : in If/Else Statements

Remember ...

== When testing for
equals in your condition



```
word = "Chocolate"
if word == "GPN":
    print("GPN is awesome!")
else:
    print("The word isn't GPN :(")
```

The code snippet is enclosed in a light gray box. Annotations include: a blue circle around the '==' operator in the 'if' statement, with a blue arrow pointing to it from the text '== When testing for equals in your condition'; a red circle around the colon at the end of the 'if' line, with a red arrow pointing to it from the text 'At end of each if line to say you have finished writing your condition'; and another red circle around the colon at the end of the 'else' line, with a red arrow pointing to it from the same text.

•
• **At end of each if line to say you have
finished writing your condition**

Functions



What is a function?

What you need to know about functions:

They are a piece of code that gets run a lot! These functions get run everytime you say their name.



What are functions?

Functions are like factories!

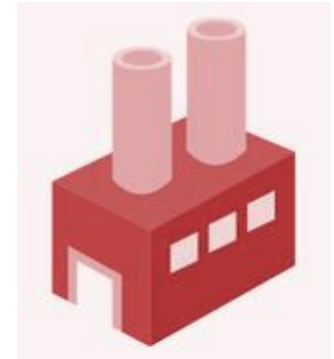
Your main factory!



Timber Mill



Metal Worker

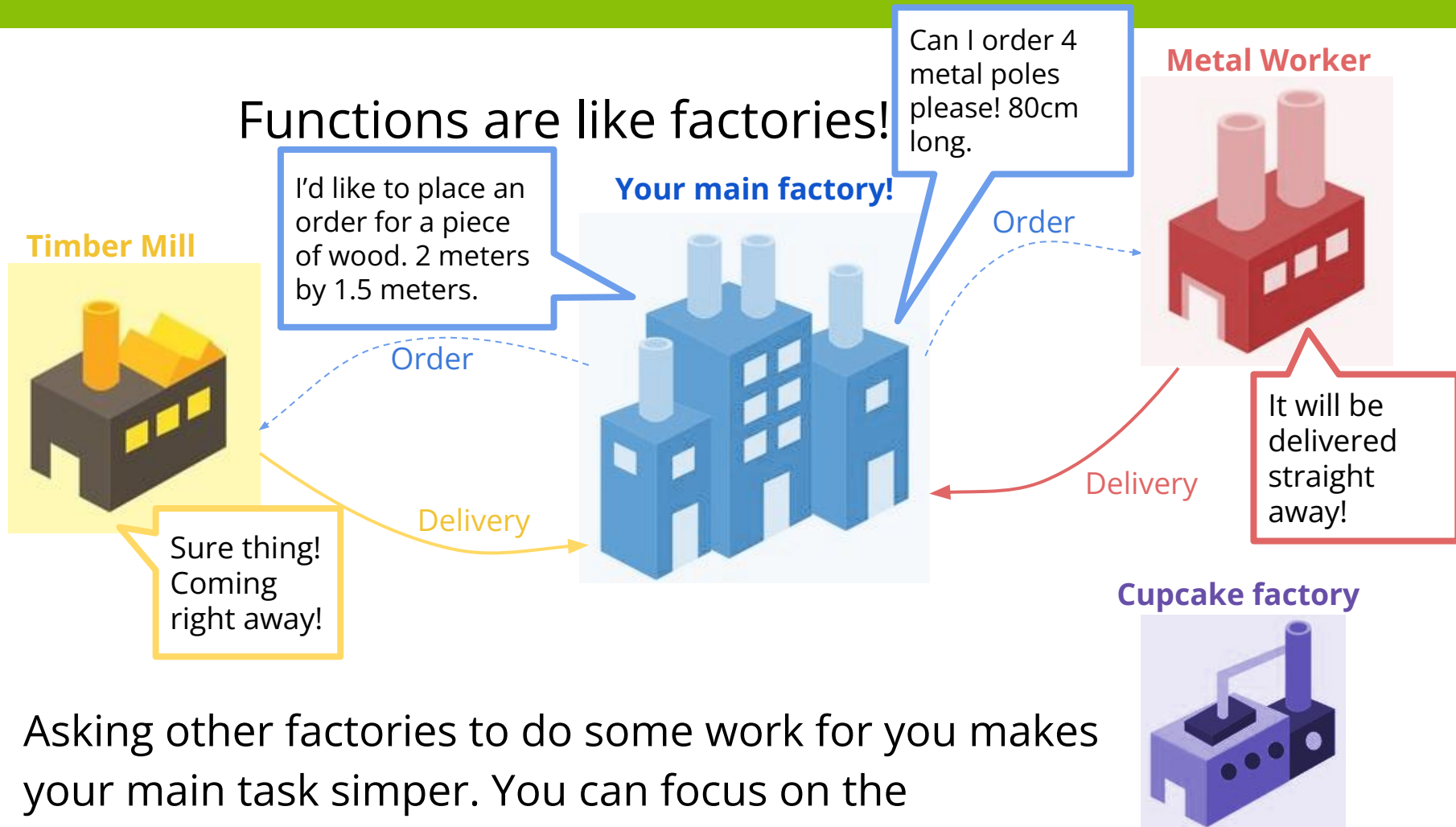


Cupcake factory



Running a factory doesn't mean doing all the work yourself, you can get other factories to help you out!

What are functions?

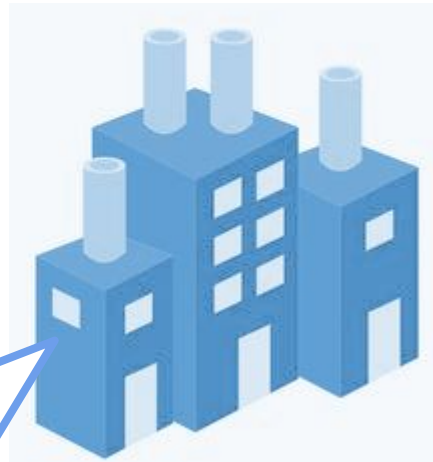


Asking other factories to do some work for you makes your main task simpler. You can focus on the assembly!

What are functions?

Functions are like factories!

Your main factory!



Timber Mill

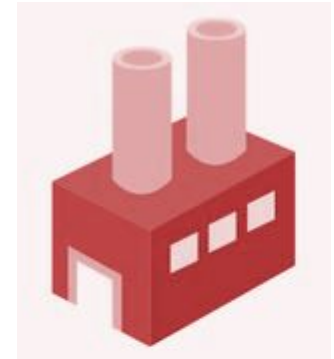


Look at this beautiful table I made!



Outsourcing made it simple!

Metal Worker



Cupcake factory



I wasn't needed for this!

Variables and functions

Variables made inside the function will stay inside the function. They are **local variables**. For example what will this do?

```
def fav_number():  
    fav_num = 20  
    print(fav_num)  
  
fav_number()  
print(fav_num)
```

Variables and functions

Variables made inside the function will stay inside the function. They are **local variables**. For example what will this do?

```
def fav_number():  
    fav_num = 20  
    print(fav_num)
```

```
fav_number()  
print(fav_num)
```

```
Traceback (most recent call last):  
  File "<stdin>", line 6, in <module>  
NameError: name 'fav_num' is not defined
```

Variables and functions

Variables created outside of a function are also **local** to their area. This means that functions cannot change them outside of themselves. What will print here?

```
def add():  
    total+=5  
  
total = 3  
add()  
print(total)
```

Variables and functions

Variables created outside of a function are also **local** to their area. This means that functions cannot change them outside of themselves. What will print here?

```
def add():  
    total+=5  
  
total = 3  
add()  
print(total)
```

3

Global variables are not affected

To fix this we can make our variable **global** by declaring it at the top of our function

```
def add():  
    global total  
    total+=5
```

```
total = 3  
add()  
print(total)
```

8

Project time!

Now go be functional.

Do the next part of the project!

Try to do Part 3

The tutors will be around to help!

Classes

Classes and Objects

What is an object?

What do you think an object is?



What is an object?

What do you think an object is?



What is an object?

What do you think an object is?



What is an object?

What do you think an object is?



What is an object?

What do you think an object is?



What is an object?

What do you think an object is?



What is an object in code?

An object is something that we know information about and that can sometimes do things



What is an object in code?

An object is something that we know information about and that can sometimes do things

Like a cat!



What is an object in code?

An object is something that we know information about and that can sometimes do things

Like a cat!



What information might we know about a cat?

What is an object in code?

An object is something that we know information about and that can sometimes do things

Like a cat!



What information might we know about a cat?

Name

What is an object in code?

An object is something that we know information about and that can sometimes do things

Like a cat!



What information might we know about a cat?

Name

Age

What is an object in code?

An object is something that we know information about and that can sometimes do things

Like a cat!



What information might we know about a cat?

Name

Age

Colour

What is an object in code?

An object is something that we know information about and that can sometimes do things

Like a cat!



What information might we know about a cat?

Name

Owner

Age

Colour

What is an object in code?

An object is something that we know information about and that can sometimes do things

Like a cat!



What information might we know about a cat?

Name

Owner

Age

Weight

Colour

What is an object in code?

An object is something that we know information about and that can sometimes do things

Like a cat!



What information might we know about a cat?

Name

Owner

Age

Weight

Colour

Microchip #

What is an object in code?

An object is something that we know information about and that can sometimes do things

Like a cat!

What things might a cat do?



What is an object in code?

An object is something that we know information about and that can sometimes do things

Like a cat!



What things might a cat do?

Meow

What is an object in code?

An object is something that we know information about and that can sometimes do things

Like a cat!



What things might a cat do?

Meow

Eat

What is an object in code?

An object is something that we know information about and that can sometimes do things

Like a cat!



What things might a cat do?

Meow

Eat

Scratch

What is an object in code?

An object is something that we know information about and that can sometimes do things

Like a cat!



What things might a cat do?

Meow

Sleep

Eat

Scratch

What is an object in code?

An object is something that we know information about and that can sometimes do things

Like a cat!



What things might a cat do?

Meow

Sleep

Eat

Purr

Scratch

What is an object in code?

An object is something that we know information about and that can sometimes do things

Like a cat!



What things might a cat do?

Meow

Sleep

Eat

Purr

Scratch

Jump

What does that look like in Python?

Let's have a look at how we might make a Cat object in Python code!



What does that look like in Python?

Let's have a look at how we might make a Cat object in Python code!

```
class Cat():  
    def __init__(self, name, age, colour):  
        self.name = name  
        self.age = age  
        self.colour = colour
```

Here we tell python that we are making a new type (or class) of object called Cat

What does that look like in Python?

Let's have a look at how we might make a Cat object in Python code!

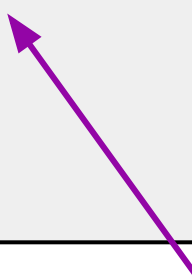
`__init__` is how we tell Python how to make a new Cat

```
class Cat():  
    def __init__(self, name, age, colour):  
        self.name = name  
        self.age = age  
        self.colour = colour
```

What does that look like in Python?

Let's have a look at how we might make a Cat object in Python code!

```
class Cat():  
    def __init__(self, name, age, colour):  
        self.name = name  
        self.age = age  
        self.colour = colour
```



Here we tell Python what information we need to know about the Cat

Note: self is special and we always need it

What does that look like in Python?

Let's have a look at how we might make a Cat object in Python code!

```
class Cat():  
    def __init__(self, name, age, colour):  
        self.name = name  
        self.age = age  
        self.colour = colour
```



Here we save the
information we got so
we can use it again

What does that look like in Python?

How do we make a new Cat?

```
class Cat():  
    def __init__(self, name, age, colour):  
        self.name = name  
        self.age = age  
        self.colour = colour  
  
emmy = Cat("Emmy", 3, "Dark brown")
```

What does that look like in Python?

What does this print out?

```
class Cat():  
    def __init__(self, name, age, colour):  
        self.name = name  
        self.age = age  
        self.colour = colour  
  
emmy = Cat("Emmy", 3, "Dark brown")  
print(emmy.name)  
print(emmy.age)  
print(emmy.colour)
```

What does that look like in Python?

What does this print out?

```
class Cat():  
    def __init__(self, name, age, colour):  
        self.name = name  
        self.age = age  
        self.colour = colour  
  
emmy = Cat("Emmy", 3, "Dark brown")  
print(emmy.name)  
print(emmy.age)  
print(emmy.colour)
```

Emmy

3

Dark Brown

I have more than 1 cat!

Emmy has a little sister, Saphira! Let's add her to our code too!

```
cat1 = Cat("Emmy", 3, "Dark brown")  
cat2 = Cat("Saphira", 1, "Grey")
```

Cat Crime!

There has been a cat crime!

One of the cats has gotten on the kitchen counter and eaten some of my lunch!

They both look innocent but they left a hair behind at the scene of the crime! Let's write some code to work out who did it



Cat Crime

Who did it??

```
cat1 = Cat("Emmy", 3, "Dark brown")
cat2 = Cat("Saphira", 1, "Grey")

hair_colour = "Grey"

if hair_colour == cat1.colour:
    print("That hair belongs to", cat1.name)
elif hair_colour == cat2.colour:
    print("That hair belongs to", cat2.name)
```

Cat Crime

Who did it??

```
cat1 = Cat("Emmy", 3, "Dark brown")
cat2 = Cat("Saphira", 1, "Grey")

hair_colour = "Grey"

if hair_colour == cat1.colour:
    print("That hair belongs to", cat1.name)
elif hair_colour == cat2.colour:
    print("That hair belongs to", cat2.name)
```

That hair belongs to Saphira

Returning from a method

Maybe we want to have a function called `get_name` that returns the name.

```
class Cat():  
    def get_name(self):  
        return self.name  
  
emmy_name = emmy.get_name()  
print("Her name is" + emmy_name + "!")
```

Her name is Emmy!

Random!

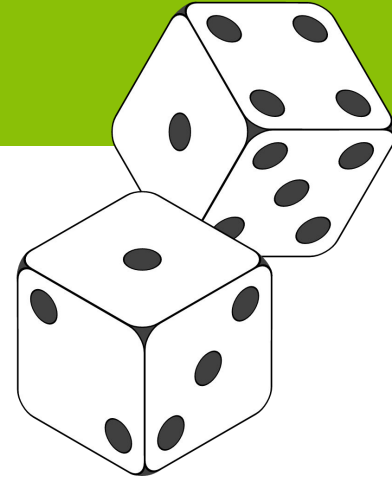


That's so random!

There's lots of things in life that are up to chance or random!



Python lets us **import** common bits of code people use! We're going to use the **random** module!



We want the computer to be random sometimes!



Using the random module

Let's choose a random number.
It's like picking a lottery number.



1. Import the random module at the top of your code like this

```
import random
```

2. Choose a random number within a range using code like this

```
random.randint(0,11)
```

It chooses a random number from the first number up to, but not including, the second!

This would choose 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 or 10

Using the random module

You can also assign your random choice to a variable

```
import random
lottery_num = random.randint(0,1000)
print(lottery_num)
```

This would choose a number between 0 and 999



Project Time!

Raaaaaaaaaandom! Can you handle that?

Let's try use it in our project!
Try to do part 4

The tutors will be around to help!

Inheritance

Super classes and Sub classes

What are we Inheriting????????

The really cool thing about classes is that we can make a thing called a Super class. But what makes it super?

We can use these "super" classes as the basis for other classes, making methods for them that the "sub" classes can inherit.

But this is much easier to explain with an analogy

Super classes

We've already talked about what we would do if we wanted to make a cat class, but what if we wanted to make a bunch of different animals?

What are some of the things all animals can do?

Super classes

We've already talked about what we would do if we wanted to make a cat class, but what if we wanted to make a bunch of different animals?

What are some of the things all animals can do?

Make noise

Walk

Play

Sleep

Eat

Super classes

We've already talked about what we would do if we wanted to make a cat class, but what if we wanted to make a bunch of different animals?

What are some of the things all animals can do?

Make noise

Walk

Play

Sleep

Eat

So wouldn't it be nice if we didn't have to write all those methods for all our classes?

Making some animal classes

Why don't we make our superclass first?

```
class Animal():  
    def __init__(self, name, age, colour):  
        self.name = name  
        self.age = age  
        self.colour = colour  
  
    def make_noise(self):  
        print("")
```



All animals make different noises so we'll be resetting this in our subclasses

Subclasses

For the subclasses there's a couple pieces of code we'll need to know.

This is how we usually make a class.

```
class Cat():
```

To make it inherit our animal class we need to add this

```
class Cat(Animal):
```

Subclasses

For the subclasses there's a couple pieces of code we'll need to know.

This is how we usually initiate a class.

```
def __init__(self, name):  
    self.name = name
```

But since we have already made one in our superclass. We just need to call it inside the subclass' init with this

```
def __init__(self, name):  
    super().__init__(name)
```

Making a cat and dog class

Why don't we make our subclasses now

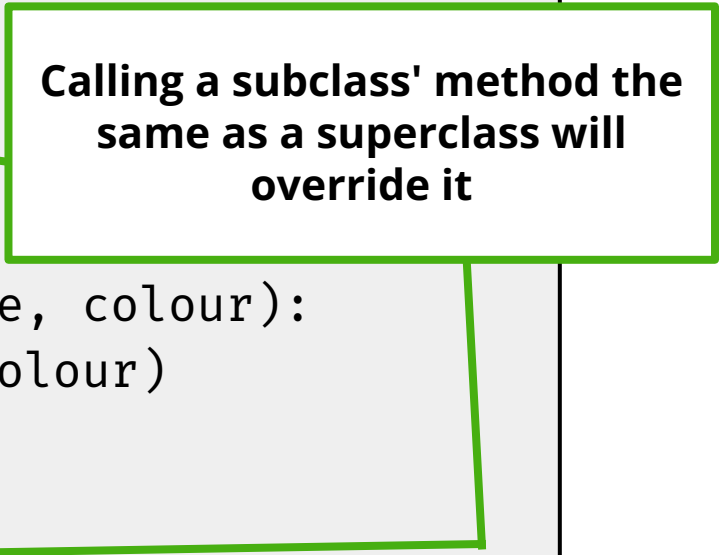
```
class Cat(Animal):  
    def __init__(self, name, age, colour):  
        super.__init__(name,age,colour)
```

```
    def make_noise(self):  
        print("Meow")
```

```
class Dog(Animal):  
    def __init__(self, name, age, colour):  
        super.__init__(name,age,colour)
```

```
    def make_noise(self):  
        print("Woof")
```

Calling a subclass' method the same as a superclass will override it



Testing our subclasses

Now let's make some animals and see what they say!

```
my_pet1 = Cat("Cleo",5,"Calico")  
  
my_pet2 = Animal("Thing",1,"Red")  
  
my_pet3 = Dog("Rufus",2,"Brown")  
  
my_pet1.make_noise()  
my_pet2.make_noise()  
my_pet3.make_noise()
```

Testing our subclasses

Now let's make some animals and see what they say!

```
my_pet1 = Cat("Cleo",5,"Calico")  
  
my_pet2 = Animal("Thing",1,"Red")  
  
my_pet3 = Dog("Rufus",2,"Brown")  
  
my_pet1.make_noise()  
my_pet2.make_noise()  
my_pet3.make_noise()
```

Meow

Woof

Project time!

Super! Now we know all about inheritance.

Now you can try to do the rest of the project!

The tutors will be around to help!